```python
# /// script
# dependencies = ["marimo", "pandas", "ortools"]
# requires-python = ">=3.14"
# ///

import marimo

__generated_with = "0.25.0"
app = marimo.App(width="medium")

with app.setup:
    import marimo as mo
    import pandas as pd
    from ortools.sat.python import cp_model


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    # TP 1.1: Criação de um horário escolar

    ## Objetivo

    Neste trabalho pretende-se gerar automaticamente um horário semanal
    para as turmas, tendo em conta as disciplinas, os professores,
    as salas disponíveis e a disponibilidade dos professores.

    O problema é modelado como um problema de satisfação de restrições
    (CSP), utilizando o solver CP-SAT do OR-Tools.

    Para além de encontrar um horário válido, pretende-se minimizar os
    "furos" nos horários dos professores e adaptar o horário quando
    existem alterações nos recursos disponíveis.
    """)
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    Os dados necessários para construir o horário são carregados a partir dos ficheiros CSV:
    """)
    return


@app.cell
def _():
    disciplinas = pd.read_csv("dados/disciplinas.csv")
    salas = pd.read_csv("dados/salas.csv")
    disponibilidade_excecoes = pd.read_csv("dados/disponibilidade_excecoes.csv")
    turmas = pd.read_csv("dados/turmas.csv")
    return disciplinas, disponibilidade_excecoes, salas, turmas


@app.cell
def _(disciplinas):
    disciplinas
    return


@app.cell
def _(salas):
    salas
    return


@app.cell
def _(disponibilidade_excecoes):
    disponibilidade_excecoes
    return
```

```python
@app.cell
def _(turmas):
    turmas
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    ## Modelação do problema

    O problema é modelado como um problema de satisfação de restrições
    (CSP) usando o CP-SAT do OR-Tools.

    Para cada possível aula criamos uma variável booleana. O valor 1
    significa que essa aula é colocada naquele dia, período e tipo de sala,
    e o valor 0 significa que não é colocada.

    As restrições R1–R8 são depois impostas sobre estas variáveis.
    """)
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    São criadas as variáveis de decisão para todas as possíveis aulas:
    """)
    return


@app.cell
def _(disciplinas, disponibilidade_excecoes, salas, turmas):
    model = cp_model.CpModel()

    dias = ["Seg", "Ter", "Qua", "Qui", "Sex"]
    periodos = [1, 2, 3, 4, 5]

    indisponiveis = set(
        zip(
            disponibilidade_excecoes["professor"],
            disponibilidade_excecoes["dia"],
            disponibilidade_excecoes["periodo"],
        )
    )

    capacidades_salas = dict(
        zip(salas["tipo"], salas["quantidade"])
    )

    x = {}

    for turma in turmas["turma"]:
        for _, disciplina in disciplinas.iterrows():

            nome_disciplina = disciplina["disciplina"]
            professor = disciplina["professor"]
            duplo = disciplina["duplo_periodo"] == "sim"
            sala_especial = disciplina["sala_especial"]


            if pd.notna(sala_especial):
                tipo_sala = sala_especial
            else:
                tipo_sala = "normal"


            if duplo:
                blocos = [(p, p + 1) for p in range(1, 5)]
            else:
                blocos = [(p,) for p in periodos]
```

```
            for dia in dias:
                for bloco in blocos:


                    professor_indisponivel = any(
                        (professor, dia, periodo) in indisponiveis
                        for periodo in bloco
                    )

                    if professor_indisponivel:
                        continue

                    chave = (
                        turma,
                        nome_disciplina,
                        dia,
                        bloco,
                        tipo_sala,
                    )

                    x[chave] = model.NewBoolVar(
                        f"x_{turma}_{nome_disciplina}_{dia}_{bloco}_{tipo_sala}"
                    )
    return dias, model, periodos, x


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    É apresentado o número de variáveis de decisão criadas:
    """)
    return


@app.cell
def _(x):
    len(x)
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    É garantida a carga semanal de cada disciplina para cada turma:
    """)
    return


@app.cell
def _(disciplinas, model, turmas, x):
    for turma_r2 in turmas["turma"]:
        for _, disciplina_r2 in disciplinas.iterrows():
            nome_r2 = disciplina_r2["disciplina"]
            carga_r2 = int(disciplina_r2["carga_semanal"])
            duplo_r2 = disciplina_r2["duplo_periodo"] == "sim"

            if duplo_r2:
                n_blocos_r2 = carga_r2 // 2
            else:
                n_blocos_r2 = carga_r2

            variaveis_r2 = [
                var
                for (t, d, dia, bloco, sala), var in x.items()
                if t == turma_r2 and d == nome_r2
            ]

            model.Add(sum(variaveis_r2) == n_blocos_r2)
    return


@app.cell(hide_code=True)
```

```
def _():
    mo.md(r"""
    É garantido que uma disciplina não ocorre mais do que uma vez por dia para cada turma:
    """)
    return


@app.cell
def _(dias, disciplinas, model, turmas, x):
    for turma_r3 in turmas["turma"]:
        for _, disciplina_r3 in disciplinas.iterrows():
            nome_r3 = disciplina_r3["disciplina"]

            for dia_r3 in dias:
                variaveis_r3 = [
                    var
                    for (t, d, dia_x, bloco, sala), var in x.items()
                    if t == turma_r3
                    and d == nome_r3
                    and dia_x == dia_r3
                ]

                model.Add(sum(variaveis_r3) <= 1)
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    É garantido que uma turma não tem duas aulas simultâneas:
    """)
    return


@app.cell
def _(dias, model, periodos, turmas, x):
    for turma_r1 in turmas["turma"]:
        for dia_r1 in dias:
            for periodo_r1 in periodos:

                variaveis_r1 = [
                    var
                    for (t, d, dia_x, bloco, sala), var in x.items()
                    if t == turma_r1
                    and dia_x == dia_r1
                    and periodo_r1 in bloco
                ]

                model.Add(sum(variaveis_r1) <= 1)
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    É garantido que um professor não leciona duas aulas simultaneamente:
    """)
    return


@app.cell
def _(dias, disciplinas, model, periodos, x):
    professores_r5 = disciplinas.set_index("disciplina")["professor"].to_dict()

    for professor_r5 in disciplinas["professor"].unique():
        for dia_r5 in dias:
            for periodo_r5 in periodos:

                variaveis_r5 = [
                    var
                    for (turma, disciplina, dia_x, bloco, sala), var in x.items()
                    if professores_r5[disciplina] == professor_r5
```

```
                    and dia_x == dia_r5
                    and periodo_r5 in bloco
                ]

                model.Add(sum(variaveis_r5) <= 1)
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    São verificadas as capacidades e os tipos de sala necessários:
    """)
    return


@app.cell
def _(dias, model, periodos, salas, x):
    capacidades_r7 = dict(
        zip(salas["sala"], salas["quantidade"])
    )

    for sala_r7, capacidade_r7 in capacidades_r7.items():

        tipo_r7 = "normal" if sala_r7 == "Sala Normal" else sala_r7

        for dia_r7 in dias:
            for periodo_r7 in periodos:

                variaveis_r7 = [
                    var
                    for (turma, disciplina, dia_x, bloco, sala_x), var in x.items()
                    if sala_x == tipo_r7
                    and dia_x == dia_r7
                    and periodo_r7 in bloco
                ]

                model.Add(sum(variaveis_r7) <= int(capacidade_r7))
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    O modelo inicial é resolvido pelo solver CP-SAT:
    """)
    return


@app.cell
def _(model):
    solver_final = cp_model.CpSolver()

    solver_final.parameters.max_time_in_seconds = 10
    solver_final.parameters.num_search_workers = 1

    status_final = solver_final.Solve(model)

    print(solver_final.StatusName(status_final))
    return (solver_final,)


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    A solução encontrada é convertida no horário H0:
    """)
    return


@app.cell
def _(solver_final, x):
```

```
    horario_resultado = []

    for (turma_h, disciplina_h, dia_h, bloco_h, sala_h), var_h in x.items():
        if solver_final.Value(var_h) == 1:
            horario_resultado.append({
                "turma": turma_h,
                "disciplina": disciplina_h,
                "dia": dia_h,
                "bloco": bloco_h,
                "sala": sala_h
            })

    horario_resultado = pd.DataFrame(horario_resultado)

    horario_resultado
    return (horario_resultado,)


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    São criadas verificações automáticas das restrições:
    """)
    return


@app.cell
def _(horario_resultado):
    def verificar_r1(horario_resultado):
        duplicados = (
            horario_resultado
            .explode("bloco")
            .groupby(["turma", "dia", "bloco"])
            .size()
        )

        return (duplicados <= 1).all()


    verificar_r1(horario_resultado)
    return (verificar_r1,)


@app.cell
def _(disciplinas, horario_resultado, turmas):
    def verificar_r2(horario_resultado, disciplinas):
        for turma in turmas["turma"]:
            for _, disciplina in disciplinas.iterrows():
                nome = disciplina["disciplina"]
                carga = int(disciplina["carga_semanal"])

                if disciplina["duplo_periodo"] == "sim":
                    carga = carga // 2

                aulas = horario_resultado[
                    (horario_resultado["turma"] == turma) &
                    (horario_resultado["disciplina"] == nome)
                ]

                if len(aulas) != carga:
                    return False

        return True


    verificar_r2(horario_resultado, disciplinas)
    return (verificar_r2,)


@app.cell
def _(horario_resultado):
    def verificar_r3(horario_resultado):
```

```
        repeticoes = (
            horario_resultado
            .groupby(["turma", "disciplina", "dia"])
            .size()
        )

        return (repeticoes <= 1).all()


    verificar_r3(horario_resultado)
    return (verificar_r3,)


@app.cell
def _(disciplinas, horario_resultado):
    def verificar_r4(horario_resultado, disciplinas):
        duplas = set(
            disciplinas.loc[
                disciplinas["duplo_periodo"] == "sim",
                "disciplina"
            ]
        )

        for _, aula in horario_resultado.iterrows():
            if aula["disciplina"] in duplas:
                bloco = aula["bloco"]

                if len(bloco) != 2 or bloco[1] != bloco[0] + 1:
                    return False

        return True


    verificar_r4(horario_resultado, disciplinas)
    return (verificar_r4,)


@app.cell
def _(disciplinas, horario_resultado):
    professor_da_disciplina_r5 = dict(
        zip(disciplinas["disciplina"], disciplinas["professor"])
    )

    def verificar_r5(horario_resultado, professor_da_disciplina_r5):
        ocupacoes = []

        for _, aula in horario_resultado.iterrows():
            professor = professor_da_disciplina_r5[aula["disciplina"]]

            for periodo in aula["bloco"]:
                ocupacoes.append(
                    (professor, aula["dia"], periodo)
                )

        ocupacoes = pd.DataFrame(
            ocupacoes,
            columns=["professor", "dia", "periodo"]
        )

        repeticoes = (
            ocupacoes
            .groupby(["professor", "dia", "periodo"])
            .size()
        )

        return (repeticoes <= 1).all()


    verificar_r5(horario_resultado, professor_da_disciplina_r5)
    return professor_da_disciplina_r5, verificar_r5
```

```python
@app.cell
def _(disciplinas, disponibilidade_excecoes, horario_resultado):
    def verificar_r6(horario_resultado, disciplinas, disponibilidade_excecoes):
        professor_da_disciplina_r6 = dict(
            zip(disciplinas["disciplina"], disciplinas["professor"])
        )

        indisponiveis_r6 = set(
            zip(
                disponibilidade_excecoes["professor"],
                disponibilidade_excecoes["dia"],
                disponibilidade_excecoes["periodo"],
            )
        )

        for _, aula in horario_resultado.iterrows():
            professor = professor_da_disciplina_r6[aula["disciplina"]]

            for periodo in aula["bloco"]:
                if (professor, aula["dia"], periodo) in indisponiveis_r6:
                    return False

        return True


    verificar_r6(
        horario_resultado,
        disciplinas,
        disponibilidade_excecoes
    )
    return (verificar_r6,)


@app.cell
def _(disciplinas, horario_resultado, salas):
    def verificar_r7(horario_resultado, disciplinas, salas):
        sala_da_disciplina_r7 = dict(
            zip(
                disciplinas["disciplina"],
                disciplinas["sala_especial"]
            )
        )

        capacidades_r7 = dict(
            zip(
                salas["sala"],
                salas["quantidade"]
            )
        )

        ocupacoes = []

        for _, aula in horario_resultado.iterrows():
            disciplina = aula["disciplina"]
            sala = aula["sala"]

            sala_especial = sala_da_disciplina_r7[disciplina]

            if pd.notna(sala_especial):
                if sala != sala_especial:
                    return False
            else:
                if sala != "normal":
                    return False


            for periodo in aula["bloco"]:
                ocupacoes.append(
                    (aula["dia"], periodo, sala)
                )

        ocupacoes = pd.DataFrame(
```

```
            ocupacoes,
            columns=["dia", "periodo", "sala"]
        )


        for (dia, periodo, sala), grupo in ocupacoes.groupby(
            ["dia", "periodo", "sala"]
        ):
            capacidade = capacidades_r7["Sala Normal"] if sala == "normal" else
capacidades_r7[sala]

            if len(grupo) > capacidade:
                return False

        return True


    verificar_r7(horario_resultado, disciplinas, salas)
    return (verificar_r7,)


@app.cell
def _(disciplinas, disponibilidade_excecoes, salas, turmas):
    def verificar_r8(disciplinas, salas, disponibilidade_excecoes, turmas):
        return (
            disciplinas is not None
            and salas is not None
            and disponibilidade_excecoes is not None
            and turmas is not None
            and len(disciplinas) > 0
            and len(salas) > 0
            and len(turmas) > 0
        )


    verificar_r8(
        disciplinas,
        salas,
        disponibilidade_excecoes,
        turmas
    )
    return (verificar_r8,)


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    As restrições R1-R8 são verificadas automaticamente sobre o horário H0:
    """)
    return


@app.cell
def _(
    disciplinas,
    disponibilidade_excecoes,
    horario_resultado,
    professor_da_disciplina_r5,
    salas,
    turmas,
    verificar_r1,
    verificar_r2,
    verificar_r3,
    verificar_r4,
    verificar_r5,
    verificar_r6,
    verificar_r7,
    verificar_r8,
):
    verificacoes = {
        "R1": verificar_r1(horario_resultado),
        "R2": verificar_r2(horario_resultado, disciplinas),
```

```
        "R3": verificar_r3(horario_resultado),
        "R4": verificar_r4(horario_resultado, disciplinas),
        "R5": verificar_r5(horario_resultado, professor_da_disciplina_r5),
        "R6": verificar_r6(
            horario_resultado,
            disciplinas,
            disponibilidade_excecoes
        ),
        "R7": verificar_r7(
            horario_resultado,
            disciplinas,
            salas
        ),
        "R8": verificar_r8(
            disciplinas,
            salas,
            disponibilidade_excecoes,
            turmas
        )
    }

    pd.DataFrame(
        list(verificacoes.items()),
        columns=["Regra", "Satisfeita"]
    )
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    ## Otimização dos furos

    Depois de encontrar uma solução que satisfaz R1–R8, acrescentamos
    uma função objetivo que minimiza o número de períodos livres entre
    a primeira e a última aula de cada professor em cada dia.
    """)
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    São criadas as variáveis que representam a ocupação dos professores em cada período:
    """)
    return


@app.cell
def _(dias, disciplinas, model, periodos, x):
    ocupacao_professor = {}

    professores_o1 = disciplinas["professor"].unique()

    for professor_o1 in professores_o1:
        for dia_o1 in dias:
            for periodo_o1 in periodos:

                variaveis_o1 = [
                    var
                    for (turma, disciplina, dia_x, bloco, sala), var in x.items()
                    if (
                        disciplinas.loc[
                            disciplinas["disciplina"] == disciplina,
                            "professor"
                        ].iloc[0] == professor_o1
                        and dia_x == dia_o1
                        and periodo_o1 in bloco
                    )
                ]

                ocupacao_professor[
```

```
                    (professor_o1, dia_o1, periodo_o1)
                ] = model.NewBoolVar(
                    f"ocupacao_{professor_o1}_{dia_o1}_{periodo_o1}"
                )

                model.Add(
                    ocupacao_professor[
                        (professor_o1, dia_o1, periodo_o1)
                    ] == sum(variaveis_o1)
                )
    return ocupacao_professor, professores_o1


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    São identificados os períodos que têm aulas antes e depois.
    """)
    return


@app.cell
def _(dias, model, ocupacao_professor, periodos, professores_o1):
    antes_o1 = {}
    depois_o1 = {}
    holes_o1 = {}

    for professor_o1_b in professores_o1:
        for dia_o1_b in dias:
            for periodo_o1_b in periodos:

                antes_o1[(professor_o1_b, dia_o1_b, periodo_o1_b)] = model.NewBoolVar(
                    f"antes_{professor_o1_b}_{dia_o1_b}_{periodo_o1_b}"
                )

                depois_o1[(professor_o1_b, dia_o1_b, periodo_o1_b)] = model.NewBoolVar(
                    f"depois_{professor_o1_b}_{dia_o1_b}_{periodo_o1_b}"
                )

                holes_o1[(professor_o1_b, dia_o1_b, periodo_o1_b)] = model.NewBoolVar(
                    f"hole_{professor_o1_b}_{dia_o1_b}_{periodo_o1_b}"
                )

                periodos_antes_o1_b = [
                    ocupacao_professor[(professor_o1_b, dia_o1_b, p)]
                    for p in periodos
                    if p < periodo_o1_b
                ]

                periodos_depois_o1_b = [
                    ocupacao_professor[(professor_o1_b, dia_o1_b, p)]
                    for p in periodos
                    if p > periodo_o1_b
                ]

                if periodos_antes_o1_b:
                    model.AddMaxEquality(
                        antes_o1[(professor_o1_b, dia_o1_b, periodo_o1_b)],
                        periodos_antes_o1_b
                    )
                else:
                    model.Add(
                        antes_o1[(professor_o1_b, dia_o1_b, periodo_o1_b)] == 0
                    )

                if periodos_depois_o1_b:
                    model.AddMaxEquality(
                        depois_o1[(professor_o1_b, dia_o1_b, periodo_o1_b)],
                        periodos_depois_o1_b
                    )
                else:
                    model.Add(
```

```
                        depois_o1[(professor_o1_b, dia_o1_b, periodo_o1_b)] == 0
                    )
    return antes_o1, depois_o1, holes_o1


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    São definidas as condições que determinam quando um período é considerado um furo:
    """)
    return


@app.cell
def _(
    antes_o1,
    depois_o1,
    dias,
    holes_o1,
    model,
    ocupacao_professor,
    periodos,
    professores_o1,
):
    for professor_o1_c in professores_o1:
        for dia_o1_c in dias:
            for periodo_o1_c in periodos:

                ocupacao_o1_c = ocupacao_professor[
                    (professor_o1_c, dia_o1_c, periodo_o1_c)
                ]

                antes_var_o1_c = antes_o1[
                    (professor_o1_c, dia_o1_c, periodo_o1_c)
                ]

                depois_var_o1_c = depois_o1[
                    (professor_o1_c, dia_o1_c, periodo_o1_c)
                ]

                hole_var_o1_c = holes_o1[
                    (professor_o1_c, dia_o1_c, periodo_o1_c)
                ]

                model.Add(hole_var_o1_c <= antes_var_o1_c)
                model.Add(hole_var_o1_c <= depois_var_o1_c)
                model.Add(hole_var_o1_c <= 1 - ocupacao_o1_c)

                model.Add(
                    hole_var_o1_c >=
                    antes_var_o1_c + depois_var_o1_c - ocupacao_o1_c - 1
                )
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    O objetivo é minimizar o número total de furos nos horários dos professores:
    """)
    return


@app.cell
def _(holes_o1, model):
    model.Minimize(
        sum(holes_o1.values())
    )
    return


@app.cell(hide_code=True)
```

```
def _():
    mo.md(r"""
    O modelo é novamente resolvido, agora com a função objetivo:
    """)
    return


@app.cell
def _(model):
    solver_o1 = cp_model.CpSolver()

    solver_o1.parameters.max_time_in_seconds = 30
    solver_o1.parameters.num_search_workers = 1

    status_o1 = solver_o1.Solve(model)

    print(solver_o1.StatusName(status_o1))
    print("Número de holes:", solver_o1.ObjectiveValue())
    return (solver_o1,)


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    A solução otimizada é convertida no horário final:
    """)
    return


@app.cell
def _(solver_o1, x):
    horario_otimizado = []

    for (turma_h1, disciplina_h1, dia_h1, bloco_h1, sala_h1), var_h1 in x.items():
        if solver_o1.Value(var_h1) == 1:
            horario_otimizado.append({
                "turma": turma_h1,
                "disciplina": disciplina_h1,
                "dia": dia_h1,
                "bloco": bloco_h1,
                "sala": sala_h1
            })

    horario_otimizado = pd.DataFrame(horario_otimizado)

    horario_otimizado
    return (horario_otimizado,)


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    O horário otimizado é novamente validado pelas restrições R1-R8:
    """)
    return


@app.cell
def _(
    disciplinas,
    disponibilidade_excecoes,
    horario_otimizado,
    professor_da_disciplina_r5,
    salas,
    turmas,
    verificar_r1,
    verificar_r2,
    verificar_r3,
    verificar_r4,
    verificar_r5,
    verificar_r6,
    verificar_r7,
```

```
    verificar_r8,
):
    verificacoes_otimizadas = {
        "R1": verificar_r1(horario_otimizado),
        "R2": verificar_r2(horario_otimizado, disciplinas),
        "R3": verificar_r3(horario_otimizado),
        "R4": verificar_r4(horario_otimizado, disciplinas),
        "R5": verificar_r5(
            horario_otimizado,
            professor_da_disciplina_r5
        ),
        "R6": verificar_r6(
            horario_otimizado,
            disciplinas,
            disponibilidade_excecoes
        ),
        "R7": verificar_r7(
            horario_otimizado,
            disciplinas,
            salas
        ),
        "R8": verificar_r8(
            disciplinas,
            salas,
            disponibilidade_excecoes,
            turmas
        )
    }

    verificacoes_otimizadas_df = pd.DataFrame(
        list(verificacoes_otimizadas.items()),
        columns=["Regra", "Satisfeita"]
    )

    verificacoes_otimizadas_df
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    São carregados os dados atualizados que serão utilizados para construir H1:
    """)
    return


@app.cell
def _(disciplinas, disponibilidade_excecoes, salas, turmas):
    disciplinas_v2 = pd.read_csv("dados_v2/disciplinas.csv")
    salas_v2 = pd.read_csv("dados_v2/salas.csv")
    disponibilidade_excecoes_v2 = pd.read_csv(
        "dados_v2/disponibilidade_excecoes.csv"
    )
    turmas_v2 = pd.read_csv("dados_v2/turmas.csv")

    print("Alterações em disciplinas:")
    print(
        disciplinas.merge(
            disciplinas_v2,
            how="outer",
            indicator=True
        ).query("_merge != 'both'")
    )

    print("\nAlterações em salas:")
    print(
        salas.merge(
            salas_v2,
            how="outer",
            indicator=True
        ).query("_merge != 'both'")
    )
```

```
    print("\nAlterações em disponibilidade:")
    print(
        disponibilidade_excecoes.merge(
            disponibilidade_excecoes_v2,
            how="outer",
            indicator=True
        ).query("_merge != 'both'")
    )

    print("\nAlterações em turmas:")
    print(
        turmas.merge(
            turmas_v2,
            how="outer",
            indicator=True
        ).query("_merge != 'both'")
    )
    return disciplinas_v2, disponibilidade_excecoes_v2, salas_v2, turmas_v2


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    É construído o modelo H1 tendo em conta os dados atualizados:
    """)
    return


@app.cell
def _(
    dias,
    disciplinas_v2,
    disponibilidade_excecoes_v2,
    periodos,
    salas_v2,
    turmas_v2,
):
    modelo_h1 = cp_model.CpModel()

    x_h1 = {}

    indisponiveis_h1 = set(
        zip(
            disponibilidade_excecoes_v2["professor"],
            disponibilidade_excecoes_v2["dia"],
            disponibilidade_excecoes_v2["periodo"],
        )
    )

    capacidades_h1 = dict(
        zip(
            salas_v2["sala"],
            salas_v2["quantidade"]
        )
    )

    for turma_h1_a in turmas_v2["turma"]:
        for _, disciplina_h1_a in disciplinas_v2.iterrows():

            nome_disciplina_h1 = disciplina_h1_a["disciplina"]
            professor_h1_a = disciplina_h1_a["professor"]
            duplo_h1_a = disciplina_h1_a["duplo_periodo"] == "sim"
            sala_especial_h1_a = disciplina_h1_a["sala_especial"]

            if pd.notna(sala_especial_h1_a):
                tipo_sala_h1_a = sala_especial_h1_a
            else:
                tipo_sala_h1_a = "normal"

            if duplo_h1_a:
                blocos_h1_a = [(p, p + 1) for p in range(1, 5)]
```

```
            else:
                blocos_h1_a = [(p,) for p in periodos]

            for dia_h1_a in dias:
                for bloco_h1_a in blocos_h1_a:

                    professor_indisponivel_h1 = any(
                        (professor_h1_a, dia_h1_a, periodo_h1_a)
                        in indisponiveis_h1
                        for periodo_h1_a in bloco_h1_a
                    )

                    if professor_indisponivel_h1:
                        continue

                    chave_h1 = (
                        turma_h1_a,
                        nome_disciplina_h1,
                        dia_h1_a,
                        bloco_h1_a,
                        tipo_sala_h1_a,
                    )

                    x_h1[chave_h1] = modelo_h1.NewBoolVar(

f"x_h1_{turma_h1_a}_{nome_disciplina_h1}_{dia_h1_a}_{bloco_h1_a}_{tipo_sala_h1_a}"
                    )

    print("Variáveis H1:", len(x_h1))
    return capacidades_h1, modelo_h1, x_h1


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    As restrições do problema são aplicadas ao novo modelo H1
    """)
    return


@app.cell
def _(
    capacidades_h1,
    dias,
    disciplinas_v2,
    modelo_h1,
    periodos,
    turmas_v2,
    x_h1,
):
    for turma_h1_r2 in turmas_v2["turma"]:
        for _, disciplina_h1_r2 in disciplinas_v2.iterrows():
            nome_h1_r2 = disciplina_h1_r2["disciplina"]
            carga_h1_r2 = int(disciplina_h1_r2["carga_semanal"])
            duplo_h1_r2 = disciplina_h1_r2["duplo_periodo"] == "sim"

            if duplo_h1_r2:
                n_blocos_h1_r2 = carga_h1_r2 // 2
            else:
                n_blocos_h1_r2 = carga_h1_r2

            variaveis_h1_r2 = [
                var_h1_r2
                for (t_h1_r2, d_h1_r2, dia_h1_r2, bloco_h1_r2, sala_h1_r2), var_h1_r2
                in x_h1.items()
                if t_h1_r2 == turma_h1_r2
                and d_h1_r2 == nome_h1_r2
            ]

            modelo_h1.Add(
                sum(variaveis_h1_r2) == n_blocos_h1_r2
            )
```

```
for turma_h1_r3 in turmas_v2["turma"]:
    for _, disciplina_h1_r3 in disciplinas_v2.iterrows():
        nome_h1_r3 = disciplina_h1_r3["disciplina"]

        for dia_h1_r3 in dias:
            variaveis_h1_r3 = [
                var_h1_r3
                for (t_h1_r3, d_h1_r3, dia_x_h1_r3, bloco_h1_r3, sala_h1_r3), var_h1_r3
                in x_h1.items()
                if t_h1_r3 == turma_h1_r3
                and d_h1_r3 == nome_h1_r3
                and dia_x_h1_r3 == dia_h1_r3
            ]

            modelo_h1.Add(
                sum(variaveis_h1_r3) <= 1
            )


for turma_h1_r1 in turmas_v2["turma"]:
    for dia_h1_r1 in dias:
        for periodo_h1_r1 in periodos:

            variaveis_h1_r1 = [
                var_h1_r1
                for (
                    t_h1_r1,
                    d_h1_r1,
                    dia_x_h1_r1,
                    bloco_h1_r1,
                    sala_h1_r1
                ), var_h1_r1 in x_h1.items()
                if t_h1_r1 == turma_h1_r1
                and dia_x_h1_r1 == dia_h1_r1
                and periodo_h1_r1 in bloco_h1_r1
            ]

            modelo_h1.Add(
                sum(variaveis_h1_r1) <= 1
            )


professores_h1_r5 = disciplinas_v2.set_index(
    "disciplina"
)["professor"].to_dict()

for professor_h1_r5 in disciplinas_v2["professor"].unique():
    for dia_h1_r5 in dias:
        for periodo_h1_r5 in periodos:

            variaveis_h1_r5 = [
                var_h1_r5
                for (
                    turma_h1_r5,
                    disciplina_h1_r5,
                    dia_x_h1_r5,
                    bloco_h1_r5,
                    sala_h1_r5
                ), var_h1_r5 in x_h1.items()
                if professores_h1_r5[disciplina_h1_r5] == professor_h1_r5
                and dia_x_h1_r5 == dia_h1_r5
                and periodo_h1_r5 in bloco_h1_r5
            ]

            modelo_h1.Add(
                sum(variaveis_h1_r5) <= 1
            )


for sala_h1_r7, capacidade_h1_r7 in capacidades_h1.items():
```

```
        tipo_h1_r7 = (
            "normal"
            if sala_h1_r7 == "Sala Normal"
            else sala_h1_r7
        )

        for dia_h1_r7 in dias:
            for periodo_h1_r7 in periodos:

                variaveis_h1_r7 = [
                    var_h1_r7
                    for (
                        turma_h1_r7,
                        disciplina_h1_r7,
                        dia_x_h1_r7,
                        bloco_h1_r7,
                        sala_x_h1_r7
                    ), var_h1_r7 in x_h1.items()
                    if sala_x_h1_r7 == tipo_h1_r7
                    and dia_x_h1_r7 == dia_h1_r7
                    and periodo_h1_r7 in bloco_h1_r7
                ]

                modelo_h1.Add(
                    sum(variaveis_h1_r7) <= int(capacidade_h1_r7)
                )

    print("Restrições H1 adicionadas.")
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    São identificadas as aulas de H0 diretamente afetadas pela alteração introduzida:
    """)
    return


@app.cell
def _(disciplinas, horario_otimizado):
    aulas_afetadas_h0 = horario_otimizado[
        (
            horario_otimizado["disciplina"].map(
                dict(zip(disciplinas["disciplina"], disciplinas["professor"]))
            ) == "Prof. Ana"
        )
        &
        (horario_otimizado["dia"] == "Sex")
        &
        (
            horario_otimizado["bloco"].apply(
                lambda bloco: 4 in bloco or 5 in bloco
            )
        )
    ]

    aulas_afetadas_h0
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    ## Replaneamento incremental de H0 para H1

    Após uma alteração nos recursos disponíveis, é necessário gerar um novo
    horário H1 que continue a respeitar as restrições R1–R8.

    Em vez de procurar apenas qualquer solução válida, usamos o horário H0
    como referência e maximizamos o número de aulas que permanecem exatamente
```

```
    no mesmo dia, período e tipo de sala.

    Desta forma, procuramos adaptar o horário à alteração introduzida,
    evitando alterações desnecessárias.
    """)
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    São identificadas as aulas de H0 que ainda podem ser preservadas em H1:
    """)
    return


@app.cell
def _(horario_otimizado, x_h1):
    preservadas_h1 = []

    for _, aula_h1_inc in horario_otimizado.iterrows():

        chave_h1_inc = (
            aula_h1_inc["turma"],
            aula_h1_inc["disciplina"],
            aula_h1_inc["dia"],
            aula_h1_inc["bloco"],
            aula_h1_inc["sala"]
        )

        if chave_h1_inc in x_h1:
            preservadas_h1.append(
                x_h1[chave_h1_inc]
            )

    print("Aulas de H0 que podem ser preservadas:", len(preservadas_h1))
    return (preservadas_h1,)


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    O objetivo é maximizar o número de aulas de H0 que permanecem inalteradas:
    """)
    return


@app.cell
def _(modelo_h1, preservadas_h1):
    modelo_h1.Maximize(
        sum(preservadas_h1)
    )
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    O modelo H1 é resolvido e é medido o tempo necessário para encontrar a solução:
    """)
    return


@app.cell
def _(modelo_h1):
    import time

    solver_h1 = cp_model.CpSolver()

    solver_h1.parameters.max_time_in_seconds = 30
    solver_h1.parameters.num_search_workers = 1
```

```
    inicio_h1 = time.perf_counter()

    status_h1 = solver_h1.Solve(modelo_h1)

    fim_h1 = time.perf_counter()

    tempo_h1 = fim_h1 - inicio_h1

    print("Estado:", solver_h1.StatusName(status_h1))
    print("Tempo H1:", tempo_h1, "segundos")
    print("Aulas preservadas:", solver_h1.ObjectiveValue())
    return solver_h1, tempo_h1, time


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    A solução encontrada é convertida no novo horário H1:
    """)
    return


@app.cell
def _(solver_h1, x_h1):
    horario_h1 = []

    for (turma_h1_r, disciplina_h1_r, dia_h1_r, bloco_h1_r, sala_h1_r), var_h1_r in
x_h1.items():
        if solver_h1.Value(var_h1_r) == 1:
            horario_h1.append({
                "turma": turma_h1_r,
                "disciplina": disciplina_h1_r,
                "dia": dia_h1_r,
                "bloco": bloco_h1_r,
                "sala": sala_h1_r
            })

    horario_h1 = pd.DataFrame(horario_h1)

    print("Número de aulas H1:", len(horario_h1))

    horario_h1
    return (horario_h1,)


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    O novo horário H1 é validado pelas restrições R1-R8:
    """)
    return


@app.cell
def _(
    disciplinas_v2,
    disponibilidade_excecoes_v2,
    horario_h1,
    salas_v2,
    turmas_v2,
    verificar_r1,
    verificar_r2,
    verificar_r3,
    verificar_r4,
    verificar_r5,
    verificar_r6,
    verificar_r7,
    verificar_r8,
):
    verificacoes_h1 = {
        "R1": verificar_r1(horario_h1),
        "R2": verificar_r2(horario_h1, disciplinas_v2),
```

```
        "R3": verificar_r3(horario_h1),
        "R4": verificar_r4(horario_h1, disciplinas_v2),
        "R5": verificar_r5(
            horario_h1,
            dict(zip(
                disciplinas_v2["disciplina"],
                disciplinas_v2["professor"]
            ))
        ),
        "R6": verificar_r6(
            horario_h1,
            disciplinas_v2,
            disponibilidade_excecoes_v2
        ),
        "R7": verificar_r7(
            horario_h1,
            disciplinas_v2,
            salas_v2
        ),
        "R8": verificar_r8(
            disciplinas_v2,
            salas_v2,
            disponibilidade_excecoes_v2,
            turmas_v2
        )
    }

    pd.DataFrame(
        list(verificacoes_h1.items()),
        columns=["Regra", "Satisfeita"]
    )
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    São contabilizadas as alterações entre os horários H0 e H1:
    """)
    return


@app.cell
def _(horario_h1, horario_otimizado):
    chaves_h0 = set(
        tuple(linha)
        for linha in horario_otimizado[
            ["turma", "disciplina", "dia", "bloco", "sala"]
        ].itertuples(index=False, name=None)
    )

    chaves_h1 = set(
        tuple(linha)
        for linha in horario_h1[
            ["turma", "disciplina", "dia", "bloco", "sala"]
        ].itertuples(index=False, name=None)
    )

    aulas_removidas = chaves_h0 - chaves_h1
    aulas_adicionadas = chaves_h1 - chaves_h0

    print("Aulas removidas de H0:", len(aulas_removidas))
    print("Aulas adicionadas em H1:", len(aulas_adicionadas))
    print("Total de alterações:", len(aulas_removidas))
    return aulas_removidas, chaves_h0


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    O horário é reorganizado numa tabela semanal para facilitar a sua leitura:
    """)
```

```
    return


@app.cell
def _(horario_otimizado):
    horario_tabela = horario_otimizado.copy()

    horario_tabela["periodo"] = horario_tabela["bloco"].apply(
        lambda bloco: "-".join(map(str, bloco))
    )

    ordem_dias = ["Seg", "Ter", "Qua", "Qui", "Sex"]

    horario_tabela["dia"] = pd.Categorical(
        horario_tabela["dia"],
        categories=ordem_dias,
        ordered=True
    )

    tabela_semanal = horario_tabela.pivot_table(
        index=["turma", "periodo"],
        columns="dia",
        values="disciplina",
        aggfunc="first",
        fill_value=""
    )

    tabela_semanal = tabela_semanal.reindex(
        columns=ordem_dias
    )

    tabela_semanal
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    São reunidas as principais métricas do replaneamento incremental:
    """)
    return


@app.cell
def _(aulas_removidas, horario_otimizado, solver_h1, tempo_h1):
    resultado_r9 = pd.DataFrame({
        "Métrica": [
            "Tempo de resolução H1 incremental",
            "Aulas em H0",
            "Aulas preservadas em H1",
            "Aulas alteradas"
        ],
        "Valor": [
            tempo_h1,
            len(horario_otimizado),
            int(solver_h1.ObjectiveValue()),
            len(aulas_removidas)
        ]
    })

    resultado_r9
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    É criado um conjunto de dados de teste com uma nova exceção de disponibilidade:
    """)
    return
```

```python
@app.cell
def _():
    import os
    import shutil

    os.makedirs("dados_teste", exist_ok=True)

    for ficheiro in [
        "disciplinas.csv",
        "salas.csv",
        "turmas.csv",
    ]:
        shutil.copy(f"dados/{ficheiro}", f"dados_teste/{ficheiro}")

    disponibilidade_teste = pd.read_csv("dados/disponibilidade_excecoes.csv")

    nova_excecao = pd.DataFrame({
        "professor": ["Prof. Ana"],
        "dia": ["Seg"],
        "periodo": [1]
    })

    disponibilidade_teste = pd.concat(
        [disponibilidade_teste, nova_excecao],
        ignore_index=True
    )

    disponibilidade_teste.to_csv(
        "dados_teste/disponibilidade_excecoes.csv",
        index=False
    )

    print("Dataset de teste criado.")
    return (disponibilidade_teste,)


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    É construído um novo modelo utilizando o conjunto de dados de teste:
    """)
    return


@app.cell
def _(dias, disciplinas, disponibilidade_teste, periodos, salas, turmas):
    modelo_teste = cp_model.CpModel()
    x_teste = {}

    indisponiveis_teste = set(
        zip(
            disponibilidade_teste["professor"],
            disponibilidade_teste["dia"],
            disponibilidade_teste["periodo"]
        )
    )

    capacidades_teste = dict(
        zip(salas["sala"], salas["quantidade"])
    )

    for turma_teste in turmas["turma"]:
        for _, disciplina_teste in disciplinas.iterrows():

            nome_disciplina_teste = disciplina_teste["disciplina"]
            professor_teste = disciplina_teste["professor"]
            duplo_teste = disciplina_teste["duplo_periodo"] == "sim"

            sala_especial_teste = disciplina_teste["sala_especial"]

            if pd.notna(sala_especial_teste):
                tipo_sala_teste = sala_especial_teste
```

```
            else:
                tipo_sala_teste = "normal"

            if duplo_teste:
                blocos_teste = [(p, p + 1) for p in range(1, 5)]
            else:
                blocos_teste = [(p,) for p in periodos]

            for dia_teste in dias:
                for bloco_teste in blocos_teste:

                    indisponivel_teste = any(
                        (professor_teste, dia_teste, periodo_teste)
                        in indisponiveis_teste
                        for periodo_teste in bloco_teste
                    )

                    if indisponivel_teste:
                        continue

                    chave_teste = (
                        turma_teste,
                        nome_disciplina_teste,
                        dia_teste,
                        bloco_teste,
                        tipo_sala_teste
                    )

                    x_teste[chave_teste] = modelo_teste.NewBoolVar(

f"x_teste_{turma_teste}_{nome_disciplina_teste}_{dia_teste}_{bloco_teste}_{tipo_sala_teste}"
                    )

    for turma_teste_r2 in turmas["turma"]:
        for _, disciplina_teste_r2 in disciplinas.iterrows():

            nome_teste_r2 = disciplina_teste_r2["disciplina"]
            carga_teste_r2 = int(disciplina_teste_r2["carga_semanal"])
            duplo_teste_r2 = disciplina_teste_r2["duplo_periodo"] == "sim"

            if duplo_teste_r2:
                n_blocos_teste_r2 = carga_teste_r2 // 2
            else:
                n_blocos_teste_r2 = carga_teste_r2

            variaveis_teste_r2 = [
                var
                for (t, d, dia, bloco, sala), var in x_teste.items()
                if t == turma_teste_r2 and d == nome_teste_r2
            ]

            modelo_teste.Add(
                sum(variaveis_teste_r2) == n_blocos_teste_r2
            )

    for turma_teste_r3 in turmas["turma"]:
        for _, disciplina_teste_r3 in disciplinas.iterrows():

            nome_teste_r3 = disciplina_teste_r3["disciplina"]

            for dia_teste_r3 in dias:

                variaveis_teste_r3 = [
                    var
                    for (t, d, dia, bloco, sala), var in x_teste.items()
                    if t == turma_teste_r3
                    and d == nome_teste_r3
                    and dia == dia_teste_r3
                ]

                modelo_teste.Add(
                    sum(variaveis_teste_r3) <= 1
```

```
            )

    for turma_teste_r1 in turmas["turma"]:
        for dia_teste_r1 in dias:
            for periodo_teste_r1 in periodos:

                variaveis_teste_r1 = [
                    var
                    for (t, d, dia, bloco, sala), var in x_teste.items()
                    if t == turma_teste_r1
                    and dia == dia_teste_r1
                    and periodo_teste_r1 in bloco
                ]

                modelo_teste.Add(
                    sum(variaveis_teste_r1) <= 1
                )

    professores_teste = disciplinas.set_index(
        "disciplina"
    )["professor"].to_dict()

    for professor_teste_r5 in disciplinas["professor"].unique():
        for dia_teste_r5 in dias:
            for periodo_teste_r5 in periodos:

                variaveis_teste_r5 = [
                    var
                    for (turma, disciplina, dia, bloco, sala), var
                    in x_teste.items()
                    if professores_teste[disciplina] == professor_teste_r5
                    and dia == dia_teste_r5
                    and periodo_teste_r5 in bloco
                ]

                modelo_teste.Add(
                    sum(variaveis_teste_r5) <= 1
                )

    for sala_teste_r7, capacidade_teste_r7 in capacidades_teste.items():

        tipo_teste_r7 = (
            "normal"
            if sala_teste_r7 == "Sala Normal"
            else sala_teste_r7
        )

        for dia_teste_r7 in dias:
            for periodo_teste_r7 in periodos:

                variaveis_teste_r7 = [
                    var
                    for (turma, disciplina, dia, bloco, sala), var
                    in x_teste.items()
                    if sala == tipo_teste_r7
                    and dia == dia_teste_r7
                    and periodo_teste_r7 in bloco
                ]

                modelo_teste.Add(
                    sum(variaveis_teste_r7) <= int(capacidade_teste_r7)
                )

    print("Modelo de teste criado.")
    return modelo_teste, x_teste


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    O modelo de teste é resolvido pelo solver CP-SAT:
    """)
```

```
    return


@app.cell
def _(modelo_teste):
    solver_teste = cp_model.CpSolver()
    solver_teste.parameters.max_time_in_seconds = 10
    solver_teste.parameters.num_search_workers = 1

    status_teste = solver_teste.Solve(modelo_teste)

    print("Estado:", solver_teste.StatusName(status_teste))
    return (solver_teste,)


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    A solução do conjunto de teste é convertida num horário:
    """)
    return


@app.cell
def _(solver_teste, x_teste):
    horario_teste = []

    for (turma_teste_r, disciplina_teste_r, dia_teste_r, bloco_teste_r, sala_teste_r),
var_teste_r in x_teste.items():
        if solver_teste.Value(var_teste_r) == 1:
            horario_teste.append({
                "turma": turma_teste_r,
                "disciplina": disciplina_teste_r,
                "dia": dia_teste_r,
                "bloco": bloco_teste_r,
                "sala": sala_teste_r
            })

    horario_teste = pd.DataFrame(horario_teste)

    print("Número de aulas:", len(horario_teste))
    horario_teste
    return (horario_teste,)


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    O horário de teste é validado pelas restrições R1-R8:
    """)
    return


@app.cell
def _(
    disciplinas,
    disponibilidade_teste,
    horario_teste,
    salas,
    turmas,
    verificar_r1,
    verificar_r2,
    verificar_r3,
    verificar_r4,
    verificar_r5,
    verificar_r6,
    verificar_r7,
    verificar_r8,
):
    verificacoes_teste = {
        "R1": verificar_r1(horario_teste),
        "R2": verificar_r2(horario_teste, disciplinas),
```

```
        "R3": verificar_r3(horario_teste),
        "R4": verificar_r4(horario_teste, disciplinas),
        "R5": verificar_r5(
            horario_teste,
            dict(zip(disciplinas["disciplina"], disciplinas["professor"]))
        ),
        "R6": verificar_r6(
            horario_teste,
            disciplinas,
            disponibilidade_teste
        ),
        "R7": verificar_r7(
            horario_teste,
            disciplinas,
            salas
        ),
        "R8": verificar_r8(
            disciplinas,
            salas,
            disponibilidade_teste,
            turmas
        )
    }

    verificacoes_teste
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    #H1 de raiz e comparação
    """)
    return


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    H1 é resolvido de raiz para permitir a comparação com o método incremental:
    """)
    return


@app.cell
def _(modelo_h1, time):
    modelo_h1.Maximize(0)

    solver_h1_raiz = cp_model.CpSolver()
    solver_h1_raiz.parameters.max_time_in_seconds = 30
    solver_h1_raiz.parameters.num_search_workers = 1

    inicio_h1_raiz = time.perf_counter()

    status_h1_raiz = solver_h1_raiz.Solve(modelo_h1)

    fim_h1_raiz = time.perf_counter()

    tempo_h1_raiz = fim_h1_raiz - inicio_h1_raiz

    print("Estado:", solver_h1_raiz.StatusName(status_h1_raiz))
    print("Tempo H1 resolvido de raiz:", tempo_h1_raiz, "segundos")
    return solver_h1_raiz, tempo_h1_raiz


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    A solução obtida de raiz é convertida num horário:
    """)
    return
```

```python
@app.cell
def _(chaves_h0, solver_h1_raiz, x_h1):
    horario_h1_raiz = []

    for (turma_raiz, disciplina_raiz, dia_raiz, bloco_raiz, sala_raiz), var_raiz in
x_h1.items():
        if solver_h1_raiz.Value(var_raiz) == 1:
            horario_h1_raiz.append({
                "turma": turma_raiz,
                "disciplina": disciplina_raiz,
                "dia": dia_raiz,
                "bloco": bloco_raiz,
                "sala": sala_raiz
            })

    horario_h1_raiz = pd.DataFrame(horario_h1_raiz)

    chaves_h1_raiz = set(
        tuple(linha)
        for linha in horario_h1_raiz[
            ["turma", "disciplina", "dia", "bloco", "sala"]
        ].itertuples(index=False, name=None)
    )

    alteracoes_raiz = chaves_h0 - chaves_h1_raiz

    print("Aulas alteradas ao resolver H1 de raiz:", len(alteracoes_raiz))
    return (alteracoes_raiz,)


@app.cell(hide_code=True)
def _():
    mo.md(r"""
    São comparados o método incremental e a resolução de H1 de raiz:
    """)
    return


@app.cell
def _(
    alteracoes_raiz,
    aulas_removidas,
    horario_otimizado,
    tempo_h1,
    tempo_h1_raiz,
):
    resultado_r9_final = pd.DataFrame({
        "Método": [
            "H1 incremental",
            "H1 de raiz"
        ],
        "Tempo (s)": [
            tempo_h1,
            tempo_h1_raiz
        ],
        "Aulas preservadas de H0": [
            len(horario_otimizado) - len(aulas_removidas),
            len(horario_otimizado) - len(alteracoes_raiz)
        ],
        "Aulas alteradas face a H0": [
            len(aulas_removidas),
            len(alteracoes_raiz)
        ]
    })

    resultado_r9_final
    return


@app.cell(hide_code=True)
def _():
```

```
    mo.md(r"""
    Como podemos ver acima, H1 incremental é masi rápido e preserva substancialmente mais aulas
 do que H1 de raiz
    """)
    return


if __name__ == "__main__":
    app.run()
```